

CSE 331

AUTOMATA AND COMPUTABILITY

Building DFA

Spring 2026

Building DFA



Typical questions

Define a DFA which recognizes the given language L .

or

Prove that the (given) language L is regular.

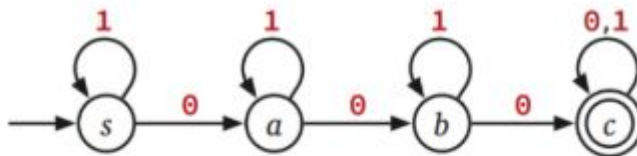
Building DFA



Example

Define a DFA which recognizes

$$L_1 = \{w \mid \text{all strings containing at least three 0s.}\}$$



- s : We've read no 0s.
- a : We've read one 0.
- b : We've read two 0s.
- c : We've read at least three 0s; accept.

Building DFA

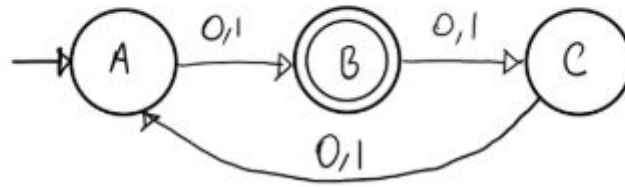


Example

Define a DFA which recognizes

$$L_2 = \{w \mid \text{the length of } w \text{ is one more than a multiple of } 3\}$$

Multiple of 3: 0, 3, 6, 9,
length: 1, 4, 7, 10, 13



Building DFA



Example

Define a DFA which recognizes

$L_3 = \{w \mid w \text{ when interpreted as a binary number, is divisible by } 3\}$

Decimal $\% 3$ = Binary $\% 3$ = remainder

$$(0)_{10} \div 3 = (0)_2 \div 3 = 0$$

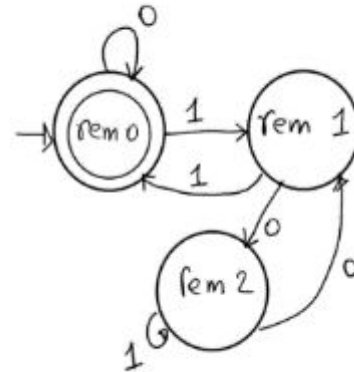
$$(1)_{10} \div 3 = (1)_2 \div 3 = 1$$

$$(2)_{10} \div 3 = (10)_2 \div 3 = 2$$

$$(3)_{10} \div 3 = (11)_2 \div 3 = 0$$

$$(4)_{10} \div 3 = (100)_2 \div 3 = 1$$

$$(5)_{10} \div 3 = (101)_2 \div 3 = 2$$



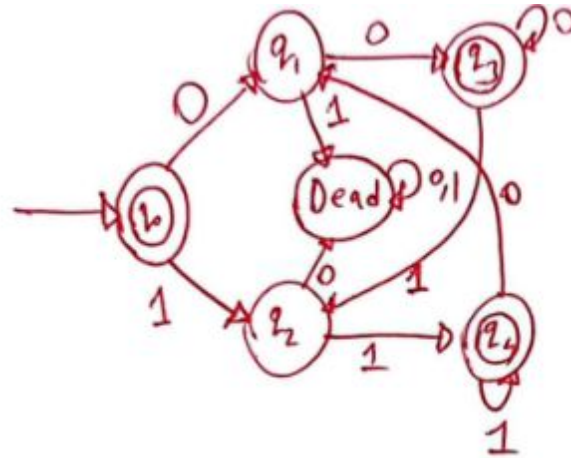
Building DFA



Example

Define a DFA which recognizes

$L_4 = \{w \mid \text{all characters are repeated at least once wherever they appear in the string}\}$



00 ✓
00011100 ✓
00100 X

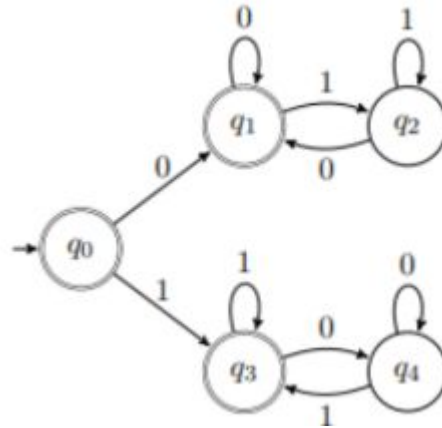
Building DFA



Example

Define a DFA which recognizes

$$L_5 = \{w \mid w \text{ contains an equal number of occurrences of the substrings } 01 \text{ and } 10\}$$



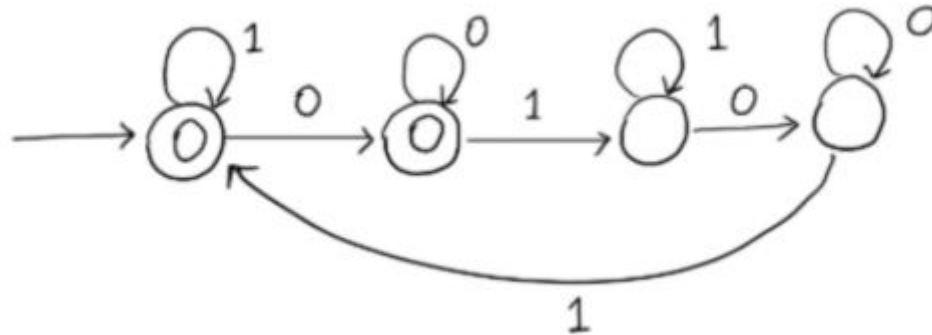
Building DFA



Example

Define a DFA which recognizes

$L_6 = \{w \mid 01 \text{ appears even number of times in } w \text{ as a substring}\}$



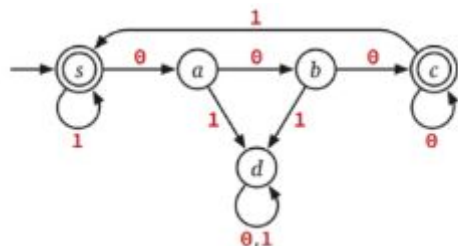
Building DFA



Example

Define a DFA which recognizes

$L_7 = \{w \mid \text{all strings in which every run of 0s has length at least 3}\}$



- s : We did not just read a 0
- a : We've read one 0 since the last 1 or the start of the string.
- b : We've read two 0 s since the last 1 or the start of the string.
- c : We've read at least three 0 s since the last 1 or the start of the string
- d : We've read the substring 01 or 001 ; reject.

Building DFA



Example

Define a DFA which recognizes

$L_8 = \{w \mid \text{all strings } w \text{ such that in every prefix of } w, \text{ the number of 0s and 1s differ by at most 1}\}$

A **prefix** of a string means:

The first few symbols of the string (starting from the beginning).

You are allowed to stop at any position.

Example

Take the string:

0101

Its prefixes are:

- ϵ (empty string)
- 0
- 01
- 010
- 0101

Notice:

A prefix always starts from the leftmost symbol.

Building DFA



Example

Define a DFA which recognizes

$L_8 = \{w \mid \text{all strings } w \text{ such that in every prefix of } w, \text{ the number of 0s and 1s differ by at most } 1\}$

Suppose you ever write two same symbols in a row:

- 00

After the prefix 00:

- #0 = 2
- #1 = 0
- Difference = 2 ✗ (not allowed)

Same problem with 11.

So you can **never repeat** a symbol consecutively.

It means while reading from left to right:

- You can never have 2 more 0s than 1s.
- You can never have 2 more 1s than 0s.

So the difference is always:

-1, 0, or 1

Building DFA

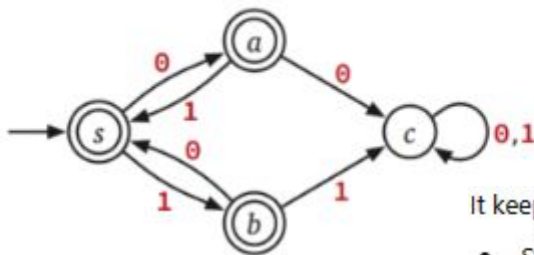


Example

Define a DFA which recognizes

$L_8 = \{w \mid \text{all strings } w \text{ such that in every prefix of } w, \text{ the number of 0s and 1s differ by at most 1}\}$

Solution: This is the same as the set of strings that alternate between 0s and 1s.



It keeps track of the current difference:

- Start state → difference 0
- After reading 0 → difference +1
- After reading 1 → difference -1
- If you repeat the same symbol → difference becomes ± 2 → go to dead state.

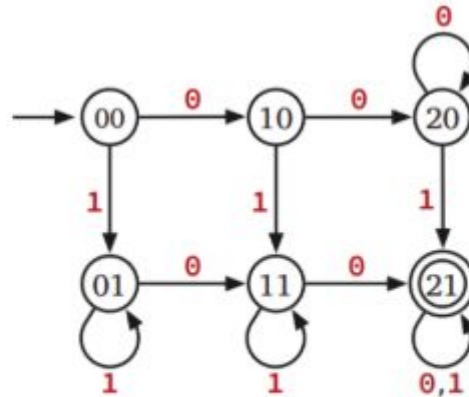
Building DFA



Example

Define a DFA which recognizes

$L_9 = \{w \mid \text{all strings containing at least two 0s and at least one 1.}\}$



Each state is labeled with a pair of integers. The first integer indicates the number of 0s read so far (up to 2), and the second indicates the number of 1s read so far (up to 1). ■



Building DFA



Remember

States are our only (computer) memory.

Design and pick states with specific roles / tasks in mind.

"Have not seen any of desired pattern yet"

"Trap state"